1 · CM Agent

Agente de community manager con IA para LinkedIn.

Propone temas desde la actualidad del sector, genera dos versiones de cada post, las lleva a un calendario y aprende en silencio el estilo editorial de la marca.

`demo` / `demo1234` — se puede probar ahora mismo.

2 · El problema

Publicar en LinkedIn de forma constante, siendo una sola persona:

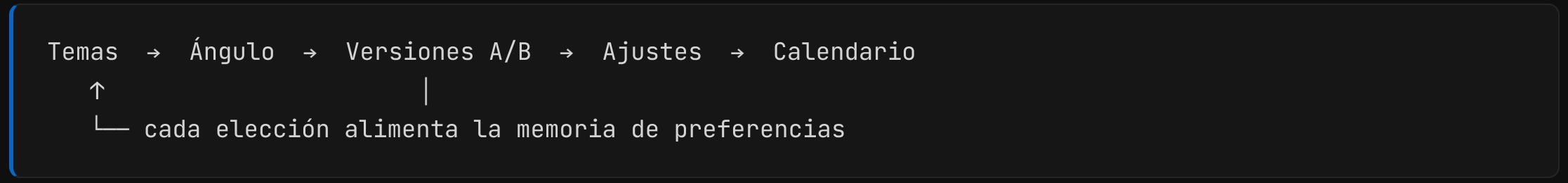
- Decidir **de qué** escribir cuesta más que escribirlo.
- Sin seguir la actualidad, el contenido queda intemporal y poco relevante.
- Una *skill* o un proyecto de IA ayudan, pero siguen siendo **instrucciones**:

el modelo las cumple casi siempre, y "casi siempre" cuela una noticia vieja.

- Lo que funcionó el mes pasado no queda registrado en ninguna parte.

3 · La propuesta

Un agente **especializado en una marca**, que ya sabe todo eso, y que **registra lo que eliges** para acertar más la próxima vez.



4 · Fase 1 · Temas

Cinco propuestas: fondo del sector y actualidad, con medio, fecha y enlace.

La regla que costó afinar: solo entra como actualidad lo publicado **hoy o ayer**, comparando la fecha de forma literal.

Si le pides "noticias recientes", un modelo llama reciente a algo de hace tres semanas. Si no hay nada fresco, completa con temas de fondo — nunca con una noticia vieja disfrazada.

5 · Fase 2 · Ángulo antes de escribir

Tres líneas: **ángulo**, **por qué ahora**, **emoción objetivo**.

Si el rumbo está mal, se corrige aquí —que es barato— y no después de leer un post entero.

6 · Fase 3 · Dos versiones para comparar

Dos versiones con ganchos **realmente distintos**, tres titulares cada una, y la estructura propia de cada formato: copy, copy + imagen (con el prompt de la imagen), carrusel, PDF o guion de video.

Elegir es la interfaz. Cada elección queda registrada.

7 · Memoria silenciosa

A partir de **tres** elecciones, el agente deduce formato, versión y estilo de titular preferidos, y propone así desde el primer intento.

Nunca lo menciona: no es un agente que explique lo que aprendió, es uno que acierta más.

Con menos de tres, no deduce nada: el patrón sería ruido.

8 · El punto de partida de este proyecto

La aplicación **ya funcionaba** en producción.

Lo que no tenía:

Pruebas	ninguna
Integración continua	no
Contenedor	no
Validación de configuración	`process.env.JWT_SECRET!`
Cabeceras de seguridad	ninguna
Marca del cliente	escrita en el código

El trabajo fue llevarla al punto en que se puede sostener y auditar.

9 · Auditoría: ocho hallazgos

El más incómodo, **H-03**:

```
let cachedMemoryCount = -1;          // ← variable del módulo = del proceso
let cachedMemorySummary = null;

if (count !== cachedMemoryCount) {   // ← la clave es el NÚMERO de entradas
  cachedMemorySummary = buildMemorySummary(memoria);
}
```

Dos personas con tres elecciones cada una recibían **las preferencias de la otra** dentro de su prompt. Funcionaba perfecto. Era una fuga de datos entre sesiones.

10 · Los otros siete

- Contraseña en claro, comparada con `===` (filtra por tiempo de respuesta).
- Acceso sin límite de intentos.
- Entrada al modelo sin techo: factura ajena.
- Inyección de prompt: el contexto podía **falsificar tarjetas** en la interfaz.
- SSRF: el servidor pedía URLs propuestas por el modelo, sin mirarlas.
- Ruta de cron abierta si faltaba su secreto.
- Cero cabeceras de seguridad.

Todos corregidos, con prueba. OWASP Top 10 y OWASP LLM recorridos en `docs/04-seguridad.md`.

11 · Pruebas

Unitarias y de integración	**383** (5 segundos)
En navegador real	**10 × 2 perfiles** (escritorio y celular)
Cobertura	98 % sentencias · 97 % funciones · 99 % líneas
Umbrales	la suite **falla** si baja

Y encontraron **once defectos** más mientras se escribían.

12 · El defecto favorito

El guion de demostración respondía **la fase equivocada** al elegir un tema.

Ninguna prueba unitaria lo vio: todas usaban frases inventadas. La interfaz manda otras ("Desarrolla el tema 1: ...").

Lo encontró el recorrido en navegador.

> **La prueba que vale es la que usa el dato real.**

El arreglo fue estructural: el contrato de mensajes vive ahora en un solo módulo, con una tabla de pruebas que recorre **todos** los mensajes posibles.

13 · Dos decisiones

El estado vive en el navegador, no en una base de datos. Parece una carencia; en la demo pública es lo contrario: dos visitantes no pueden verse nada porque no hay nada compartido que ver.

El agente tiene presupuesto diario, y al agotarse no se muere. Responde con un guion local y lo declara en una cabecera. Cualquiera puede probar la aplicación completa sin que llegue una factura sorpresa.

14 · Cómo se construyó

Dirigiendo a un agente de IA, que es la competencia que enseña el máster. Ni se escribió a mano línea por línea, ni lo hizo la IA sola.

LO DECIDÍ YO	LO EJECUTÓ EL AGENTE
Qué proyecto se entrega, y cuál no	Arquitectura, código y pruebas
Que nada lleve datos de cliente	La auditoría de seguridad
Que no se gaste dinero	La documentación
Qué se corrige y qué queda como deuda	El montaje del vídeo

Descarté dos proyectos antes que este: uno que el evaluador no habría podido probar, y otro que exponía datos reales. **Elegí el evaluable por encima del vistoso.**

15 · Infraestructura

```
cada cambio  →  lint  →  tipos  →  pruebas + cobertura
                  →  construcción
                  →  recorrido en navegador
                  →  imagen Docker levantada y comprobada
```

Sin gastar un token: la integración continua corre en modo de demostración.

16 · Gracias

Repositorio · aplicación publicada · usuario de prueba en el formulario de entrega.

La documentación técnica completa está en `docs/`: especificación con historias y criterios, requisitos con umbral, diseño con alternativas y deuda reconocida, auditoría de seguridad, estrategia de pruebas y trabajo con IA.